

# Złożoność obliczeniowa i rekurencja

Matura rozszerzona z informatyki – Sesja 4

Kacper Gębusia

- 1 Czym jest złożoność obliczeniowa
- 2 Notacja  $O$  i klasy złożoności
- 3 Analiza kodu, przypadek pesymistyczny/oczekiwany
- 4 Rekurencja
- 5 Ćwiczenia

- 1 Nazwij bramkę logiczną przedstawioną na schemacie poniżej.



- 2 Rozpisz tablicę prawdy dla operatora NOR.
- 3 Rozpisz tablicę prawdy dla wyrażenia  $(a \vee \neg b) \wedge c$ .
- 4 Co wypisze `print(13 & 6, 13 | 6, 13 ^ 6)`?

# Czym jest złożoność obliczeniowa

Dwa poprawne algorytmy rozwiązujące ten sam problem mogą różnić się ogromnie szybkością. **Złożoność obliczeniowa** opisuje, jak rośnie koszt algorytmu wraz z **rozmiarem danych**  $n$ .

- **Złożoność czasowa** – ile **operacji elementarnych** wykona algorytm
- **Złożoność pamięciowa** – ile **dodatkowej pamięci** potrzebuje algorytm

Nie mierzymy czasu w sekundach (zależy od komputera, języka, obciążenia) – liczymy operacje jako funkcję  $n$ .

**Operacja dominująca** – operacja wykonywana najczęściej (np. porównanie w najgłębszej pętli). Wystarczy policzyć, ile razy się wykona.

## Przykład 1 – szukanie maksimum

```
def maks(a):  
    m = a[0]  
    for i in range(1, len(a)):  
        if a[i] > m:          # operacja dominująca: porównanie  
            m = a[i]  
    return m
```

Dla listy  $n$  elementów porównanie wykona się  $n - 1$  razy  $\rightarrow$  złożoność czasowa  $O(n)$ .  
Pamięć dodatkowa to jedna zmienna  $m \rightarrow$  złożoność pamięciowa  $O(1)$ .

Zapis  $f(n) = O(g(n))$  oznacza (nieformalnie): dla dużych  $n$  funkcja  $f$  rośnie **nie szybciej niż**  $g$ , z dokładnością do stałej. Interesuje nas tylko **dominujący składnik**.

## Zasady upraszczania:

- pomijamy stałe współczynniki:  $3n = O(n)$
- pomijamy składniki niższego rzędu:  $n^2 + 5n + 7 = O(n^2)$
- $\frac{n(n-1)}{2} = O(n^2)$ ,  $n + \log n = O(n)$ ,  $2^n + n^{10} = O(2^n)$

# Klasy złożoności

| Klasa                 | Zapis         | Typowy przykład                         |
|-----------------------|---------------|-----------------------------------------|
| stała                 | $O(1)$        | dostęp do elementu listy $a[i]$         |
| logarytmiczna         | $O(\log n)$   | wyszukiwanie binarne                    |
| liniowa               | $O(n)$        | szukanie maksimum, wyszukiwanie liniowe |
| liniowo-logarytmiczna | $O(n \log n)$ | sortowanie przez scalanie               |
| kwadratowa            | $O(n^2)$      | sortowanie bąbelkowe                    |
| sześcienna            | $O(n^3)$      | trzy zagnieżdżone pętle                 |
| wykładnicza           | $O(2^n)$      | naiwne rekurencyjne Fibonacci           |
| silniowa              | $O(n!)$       | sprawdzanie wszystkich permutacji       |

# Jak szybko rosną funkcje

| $n$  | $\log_2 n$ | $n \log_2 n$     | $n^2$     | $n^3$     | $2^n$                        | $n!$                         |
|------|------------|------------------|-----------|-----------|------------------------------|------------------------------|
| 10   | 3,3        | 33               | 100       | 1 000     | 1 024                        | 3 628 800                    |
| 100  | 6,6        | 664              | 10 000    | 1 000 000 | $\approx 1,3 \cdot 10^{30}$  | $\approx 9,3 \cdot 10^{157}$ |
| 1000 | 10         | $\approx 9\,966$ | 1 000 000 | $10^9$    | $\approx 1,1 \cdot 10^{301}$ | astronomiczna                |

# Dlaczego to ma znaczenie w praktyce

Założmy orientacyjnie, że Python wykonuje ok.  $10^7$  prostych operacji na sekundę:

| $n$    | $O(n)$  | $O(n \log n)$ | $O(n^2)$     | $O(n^3)$      |
|--------|---------|---------------|--------------|---------------|
| $10^4$ | 0,001 s | 0,013 s       | 10 s         | ok. 28 godzin |
| $10^5$ | 0,01 s  | 0,17 s        | ok. 17 minut | ok. 3 lata    |

**Wniosek:** przy większych danych algorytm  $O(n^2)$  bywa za wolny – to wskazówka, kiedy szukać sprytniejszego rozwiązania (sortowanie + wyszukiwanie binarne, słowniki, sito).

- instrukcje jedna po drugiej  $\rightarrow$  złożoności się **dodają** (zostaje największa)
- pętle zagnieżdżone  $\rightarrow$  złożoności się **mnożą**
- pętla, w której zmienna jest **dzielona/mnożona przez 2**  $\rightarrow \log n$  powtórzeń

## Przykłady 2-3 – pętle

```
# Przykład 2 - jedna pętla
s = 0
for i in range(n):
    s += i
```

$n$  powtórzeń  $\rightarrow O(n)$ .

```
# Przykład 3 - dwie zagnieżdżone pętle
for i in range(n):
    for j in range(n):
        licznik += 1
```

$n \cdot n$  powtórzeń  $\rightarrow O(n^2)$ .

## Przykłady 4-5 – pętle zależne i sekwencyjne

```
# Przykład 4 - pętla wewnętrzna zależna od zewnętrznej
for i in range(n):
    for j in range(i):
        licznik += 1
```

Powtórzeń:  $0 + 1 + \dots + (n - 1) = \frac{n(n-1)}{2} \rightarrow O(n^2)$

```
# Przykład 5 - pętle jedna po drugiej
for i in range(n):
    x += 1
for i in range(n):
    for j in range(n):
        y += 1
```

$n + n^2 \rightarrow O(n^2)$

## Przykłady 6–7 – dzielenie przez 2

# Przykład 6 - dzielenie przez 2

```
while n > 1:
```

```
    n //= 2
```

Dla  $n = 1000$ :  $1000 \rightarrow 500 \rightarrow 250 \rightarrow 125 \rightarrow 62 \rightarrow 31 \rightarrow 15 \rightarrow 7 \rightarrow 3 \rightarrow 1$ , czyli 9 powtórzeń  $\rightarrow O(\log n)$ .

# Przykład 7 - pętla z pętlą logarytmiczną

```
for i in range(n):
```

```
    j = 1
```

```
    while j < n:
```

```
        j *= 2
```

$n \cdot \log n \rightarrow O(n \log n)$

# Pesymistyczna, oczekiwana i optymistyczna

Ten sam algorytm może działać różnie dla różnych danych o tym samym rozmiarze  $n$ :

- **pesymistyczna** – dla **najgorszych** możliwych danych
- **oczekiwana (średnia)** – uśredniona po wszystkich danych
- **optymistyczna** – dla najlepszych danych

## Przykład 8 – wyszukiwanie liniowe w liście $n$ -elementowej:

- optymistycznie:  $O(1)$  (element pierwszy)
- pesymistycznie:  $O(n)$  (element ostatni/brak)
- oczekiwanie: średnio  $\frac{n+1}{2}$  porównań, też  $O(n)$

Na maturze, gdy pytanie nie precyzuje przypadku, podajemy złożoność **pesymistyczną**.

# Rekurencja – definicja i przykład

**Rekurencja** – funkcja **wywołuje samą siebie** dla mniejszego problemu. Potrzebne są dwa elementy:

- **warunek stopu** – zwraca wynik bez dalszych wywołań
- **krok rekurencyjny** – wywołanie z argumentem bliższym przypadkowi bazowemu

**Przykład 9 – silnia:**  $n! = n \cdot (n - 1)!$ ,  $0! = 1$

```
def silnia(n):  
    if n == 0:                # warunek stopu  
        return 1  
    return n * silnia(n - 1)  # krok rekurencyjny
```

## Śledzenie silnia(4)

Wywołania „schodzą w dół”, wyniki „wracają w górę”:

```
silnia(4) = 4 * silnia(3)
           silnia(3) = 3 * silnia(2)
                   silnia(2) = 2 * silnia(1)
                           silnia(1) = 1 * silnia(0)
                                   silnia(0) = 1
```

wyniki wracaja: silnia(1)=1 silnia(2)=2 silnia(3)=6 silnia(4)=24

Każde wywołanie „czeka” na wynik kolejnego – komputer trzyma je na **stosie wywołań** (pamięć!).

# Złożoność silni: rekurencja vs iteracja

- Liczba wywołań  $\text{silnia}(n)$ :  $n + 1 \rightarrow$  **czasowa**  $O(n)$
- Na stosie czeka jednocześnie  $n + 1$  wywołań  $\rightarrow$  **pamięciowa**  $O(n)$
- Zależność:  $T(n) = T(n - 1) + 1$ ,  $T(0) = 1 \Rightarrow T(n) = n + 1 = O(n)$

**Wersja iteracyjna** – ta sama złożoność czasowa, ale pamięciowa  $O(1)$ :

```
def silnia_iter(n):  
    wynik = 1  
    for i in range(2, n + 1):  
        wynik *= i  
    return wynik
```

# Ograniczenie głębokości rekurencji

Domyślnie Python dopuszcza ok. **1000** zagnieżdżonych wywołań. `silnia(5000)` zakończy się błędem `RecursionError`.

Ten sam błąd wystąpi, gdy **zapomnimy o warunku stopu** albo krok rekurencyjny nie zbliża się do przypadku bazowego.

|        | Rekurencja                                                  | Iteracja      |
|--------|-------------------------------------------------------------|---------------|
| Zapis  | często krótszy, naturalny dla problemów „dziel i zwyciężaj” | prosta pętla  |
| Pamięć | stos, zwykle $O(\text{głębokość})$                          | zwykle $O(1)$ |
| Ryzyko | <code>RecursionError</code> dla dużych $n$                  | brak          |

# Szybkie potęgowanie

Zamiast  $a^n = a \cdot a^{n-1}$  ( $n$  wywołań) korzystamy z  $a^n = (a^{n/2})^2$  dla parzystych  $n$ :

```
def potega(a, n):  
    if n == 0:  
        return 1  
    if n % 2 == 0:  
        p = potega(a, n // 2)  
        return p * p  
    return a * potega(a, n - 1)
```

`potega(2, 10)`: wywołania dla  $n = 10, 5, 4, 2, 1, 0$ , wyniki:  $1 \rightarrow 2 \rightarrow 4 \rightarrow 16 \rightarrow 32 \rightarrow 1024$ . Liczba wywołań rośnie logarytmicznie  $\rightarrow O(\log n)$ .

# Ćwiczenia na zajęciach (1/3)

1. Podaj złożoność czasową (notacja  $O$ ) i krótko uzasadnij:

```
# a)
for i in range(n):
    for j in range(n):
        x += 1
```

```
# b)
i = 1
while i < n:
    i *= 2
```

```
# c)
for i in range(n):
    for j in range(n):
        for k in range(10):
            x += 1
```

```
# d)
for i in range(n):
    x += 1
    for j in range(n * n):
        y += 1
```

2. Ile razy wykona się `c += 1`?

# a) (`n = 10`)

```
for i in range(1, n+1):  
    for j in range(1, i+1):  
        c += 1
```

# b)

```
i = 1  
while i < 100:  
    c += 1  
    i *= 2
```

3. Uprość do klasy złożoności:

a)  $4n^3 + 100n^2 + 7$    b)  $n + \log n$    c)  $2^n + n^{10}$    d)  $\frac{n^2+n}{2}$

## Ćwiczenia na zajęciach (3/3)

- 4 Co zwróci funkcja? Prześledź wywołania.
  - a)  $g(n): 'n==0' \rightarrow 0$ , inaczej  $'n + g(n-1)'$ ; policz  $g(4)$ .
  - b)  $h(n): 'n<10' \rightarrow 'n'$ , inaczej  $'n\%10 + h(n//10)'$ ; policz  $h(2749)$ .
  - c)  $k(n): 'n==0' \rightarrow 0$ , inaczej  $'k(n//2) + n\%2'$ ; policz  $k(13)$ .
- 5 Napisz rekurencyjną potega(a, n) (naiwną) i podaj jej złożoność.
- 6 Uszereguj od najwolniej do najszybciej rosnącej:  $n^2$ ,  $2^n$ ,  $n \log n$ ,  $n$ ,  $\log n$ ,  $n!$
- 7 Dlaczego silnia(5000) kończy się błędem i jak tego uniknąć?

## 1. Podaj złożoność czasową:

|                                                                                                         |                                                                                  |
|---------------------------------------------------------------------------------------------------------|----------------------------------------------------------------------------------|
| <pre># a) for i in range(n):     for j in range(n):         for k in range(n):             s += 1</pre> | <pre># b) for i in range(n):     j = n     while j &gt; 0:         j //= 2</pre> |
| <pre># c) i = n while i &gt; 0:     i -= 3</pre>                                                        | <pre># d) def f(a):     return a[0] + a[len(a) - 1]</pre>                        |

**2.** Ile razy wykona się `c += 1`?

|                       |                      |               |
|-----------------------|----------------------|---------------|
| # a)                  | # b)                 | # c)          |
| for i in range(1, 8): | for i in range(5):   | i = 200       |
| for j in range(1, 8): | for j in range(i+1): | while i >= 1: |
| c += 1                | c += 1               | c += 1        |
|                       |                      | i //= 2       |

**3.** Prześledź i podaj wynik:

a) `f(n): 'n==0'→1, inaczej '2*f(n-1)'`; policz `f(5)`.

b) `m(a,b): 'b==0'→'a', inaczej 'm(b, a%b)'`; policz `m(48,18)`. Co oblicza `m`?

- 4 Napisz rekurencyjną `liczba_cyfr(n)` ( $n \geq 1$ ) zwracającą liczbę cyfr  $n$ . Policz `liczba_cyfr(2749)` i podaj złożoność.
- 5 Prześledź `potega(2, 10)` (szybkie potęgowanie). Ile wywołań wykonano, a ile wersja naiwna?
- 6 Wyjaśnij różnicę: złożoność pesymistyczna a oczekiwana. Ile porównań (najgorszy/średnio) wykona wyszukiwanie liniowe w liście 100 elementów?
- 7 Algorytm A:  $O(n^2)$ , algorytm B:  $O(n \log n)$ . Ile czasu zajmie każdy dla  $n = 10^4$  przy  $10^7$  operacji/s?
- 8 Co jest nie tak z `def f(n): return n * f(n - 1)`? Jak naprawić, żeby liczyła silnię?

Dziękuję za uwagę!